# Software Requirements Specification (SRS)

Academic Performance Analytics and Result Management System (APARMS)

| | |
|---|---|
| Project Type | Web-Based Application |
| Version | 1.0 |
| Purpose | Academic Project |

## 1. Introduction

### 1.1 Purpose

The Academic Performance Analytics and Result Management System (APARMS) is a web-based system designed to manage student academic records and provide meaningful analysis of academic performance. The system will allow administrators, faculty members, and students to perform different activities based on their roles. It will automate result calculation, reduce manual errors, provide academic analytics through charts and dashboards, and make student performance information easier to access.

### 1.2 Scope

The system will provide student management, faculty management, subject and semester management, marks/result management, automatic grade calculation, SGPA/CGPA calculation, student performance analytics, dashboards, result reports, authentication, and role-based access.

The initial version will focus on academic results and performance analytics. Attendance management, fee management, timetable management, and AI-based prediction are outside the scope of Version 1.0.

## 2. Stakeholders

### 2.1 Administrator

• Add, update, deactivate, and manage students.

• Add and manage faculty members.

• Add and manage subjects and semesters.

• Assign subjects to faculty.

• Manage academic records.

### 2.2 Faculty

• View assigned subjects.

• Enter and update marks.

• View student performance.

• Analyze class and subject performance.

### 2.3 Student

• View personal profile.

• View semester results.

• View subject-wise marks and grades.

• View SGPA/CGPA.

• View performance analytics.

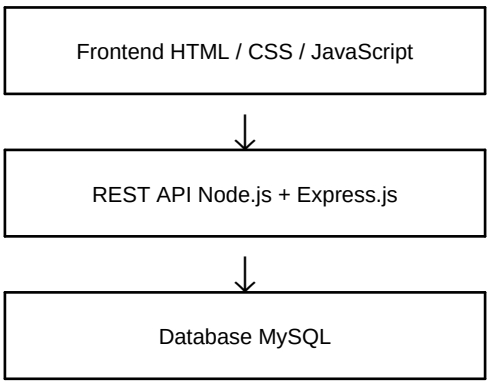
• View result reports.

# 3. Overall Description

## 3.1 Product Perspective

The system will follow a client-server architecture. The frontend communicates with a backend REST API, which processes requests and interacts with the MySQL database.

### System Architecture Flow Diagram




The frontend sends requests to the backend through REST APIs. The backend validates requests, applies business logic, and reads or writes data in MySQL.

# 4. Functional Requirements

### FR-01: User Authentication

The system shall allow users to log in using valid credentials, identify their role, redirect them to the appropriate dashboard, and prevent unauthorized access.

### FR-02: Student Management

Administrators shall be able to add, update, deactivate, search, and view student records.

### FR-03: Faculty Management

Administrators shall be able to add, update, deactivate, and assign subjects to faculty members.

### FR-04: Subject Management

Administrators shall be able to add, edit, deactivate, and manage subject codes, names, credits, and semester assignments.

### FR-05: Marks Management

Authorized faculty shall be able to enter and update marks. The system shall validate marks and prevent invalid values.

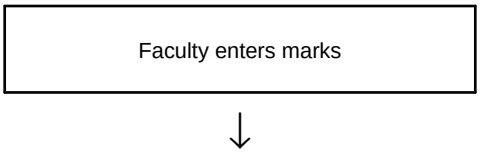
# 5. Result Management

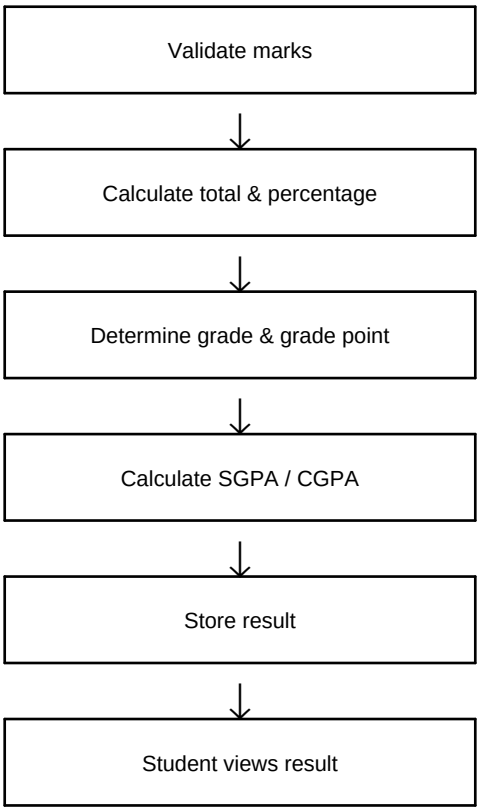
The system shall automatically calculate total marks, percentage, grade, grade point, SGPA, CGPA, and pass/fail status.

Example: Marks = 85 $\rightarrow$ Grade = A $\rightarrow$ Grade Point = 9. The exact grade boundaries will be configurable according to the institution's grading scheme.

SGPA = $\Sigma$(Credit × Grade Point) / $\Sigma$(Credits). The CGPA calculation will follow the institution's defined academic rules.

### Result Processing Flow Diagram






# 6. Performance Analytics

## 6.1 Student-Level Analytics

• Semester-wise SGPA.

• Overall CGPA.

• Subject-wise marks.

• Subject-wise grades.

• Performance trends.

## 6.2 Faculty-Level Analytics

• Class average.

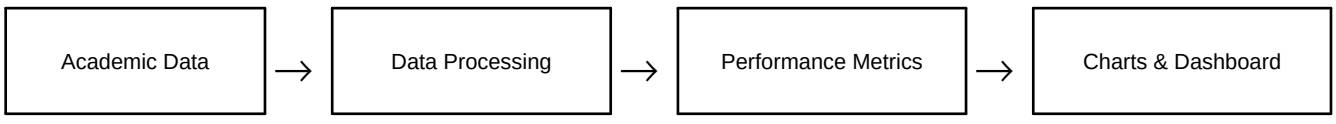
• Highest and lowest marks.

• Pass percentage.

• Fail percentage.

• Subject performance.

### Analytics Flow Diagram




# 7. Dashboard Requirements

## 7.1 Student Dashboard

The dashboard should display student information, current SGPA, CGPA, subject performance, and semester performance charts.

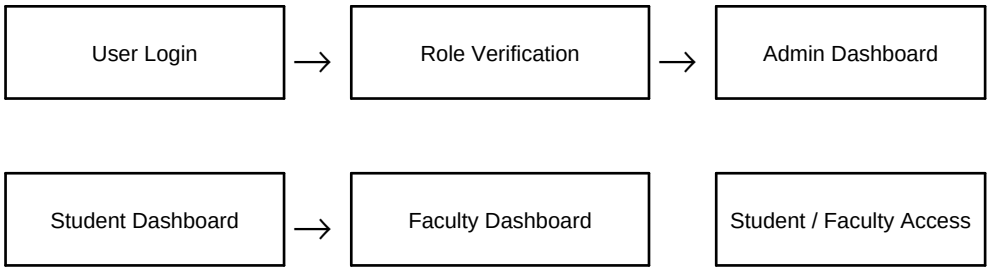
### 7.2 Faculty Dashboard

The dashboard should display assigned subjects, number of students, average marks, pass percentage, and performance charts.

### 7.3 Admin Dashboard

The dashboard should display total students, total faculty, total subjects, total semesters, and links to student, faculty, subject, and result management.

### Role-Based Dashboard Flow Diagram

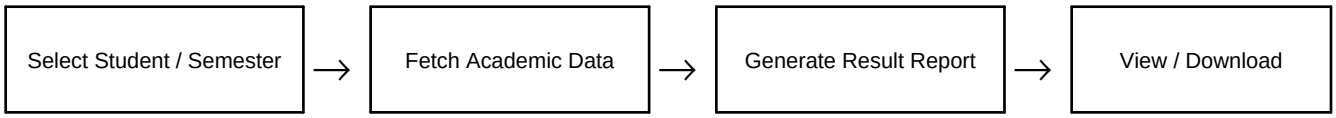



Role-based authorization determines which dashboard and operations a logged-in user can access.

## 8. Report Management

The system shall allow users to view academic reports containing student name, enrollment number, branch, semester, subject-wise credits, marks, grades, total marks, percentage, SGPA, CGPA, and result status. Downloadable PDF reports may be added during implementation.

### Report Generation Flow Diagram




## 9. Non-Functional Requirements

### Security

Passwords must not be stored as plain text. Authentication and role-based authorization shall be implemented securely.

### Performance

Dashboard pages should load quickly and database queries should be reasonably optimized.

### Usability

The interface should be simple, responsive, and provide clear error messages.

### Reliability

The system should prevent invalid data and maintain accurate result calculations.

### Maintainability

The application should use modular components for frontend, backend, authentication, analytics, and result calculation.

## 10. Technology Requirements

| Layer | Technology |
|---|---|
| Frontend | HTML, CSS, JavaScript, Bootstrap, Chart.js |
| Backend | Node.js, Express.js |

| Layer | Technology |
|---|---|
| Database | MySQL |
| Development | VS Code, Git, GitHub, Postman, MySQL Workbench |

## 11. Hardware Requirements

• Minimum: 4 GB RAM, dual-core processor, 10 GB available storage, and internet connection.

• Recommended: 8 GB or more RAM, modern Intel/AMD processor, SSD, and stable internet.

## 12. Software Requirements

VS Code for development; Node.js for backend runtime; Express.js for the backend framework; MySQL and MySQL Workbench for database management; Git/GitHub for version control; Postman for API testing; and Chrome or Edge for browser testing.

## 13. Database Requirements

The initial database will contain entities such as Users, Students, Faculty, Subjects, Semesters, Marks, and Results.

### Database Relationship Flow Diagram

Core relationships: User records authenticate Students and Faculty; Students receive Marks; Marks are associated with Subjects; academic marks are used to produce Results.

## 14. System Constraints

• Users must have valid credentials.

• Internet/network connectivity may be required for deployment.

• Only authorized faculty can modify marks.

• The grading system depends on the institution's defined grading rules.

• Version 1.0 focuses on academic performance and results rather than complete college management.

## 15. Future Enhancements

• Attendance management.

• AI-based performance prediction.

• At-risk student identification.

• Email/SMS notifications.

• Parent dashboard.

• Course recommendation.

• Automated report generation.

• Placement-performance correlation.

• Mobile application.

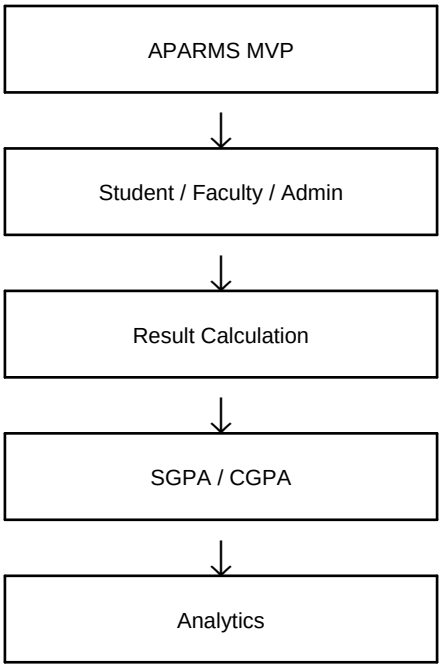
• Advanced data analytics.

# 16. MVP Definition

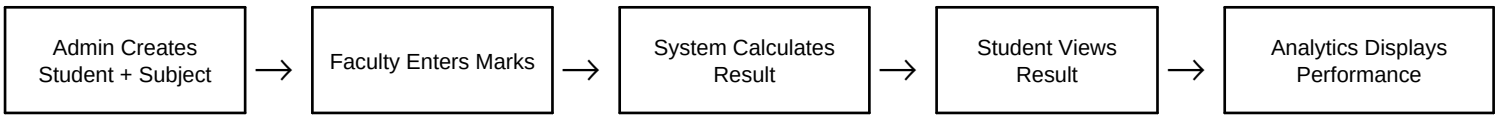
The first working version should contain Student, Faculty, and Admin modules; result calculation; SGPA/CGPA; and basic analytics.

**Core workflow:** Admin creates student and subject → Faculty enters marks → System calculates result → Student views result → Analytics displays performance.

### MVP System Flow Diagram

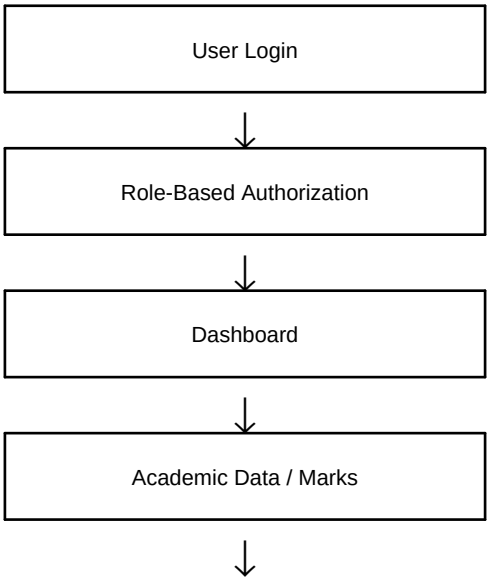


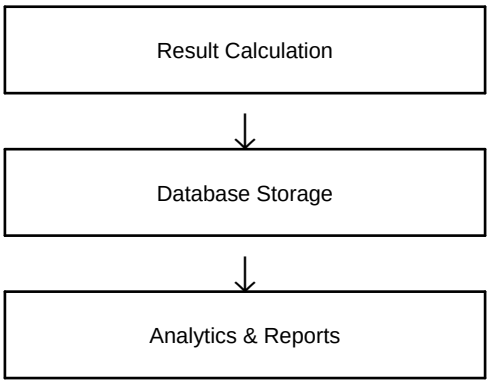

### Detailed MVP Workflow




**Development principle:** Complete this MVP end-to-end before adding future enhancements such as attendance, AI prediction, notifications, or mobile support.

# 17. Complete System Flow Summary






This represents the complete high-level workflow of the proposed Academic Performance Analytics and Result Management System.